COSC 2633/2637
Big Data Processing

Lecture 5

MapReduce Clustering and Programming

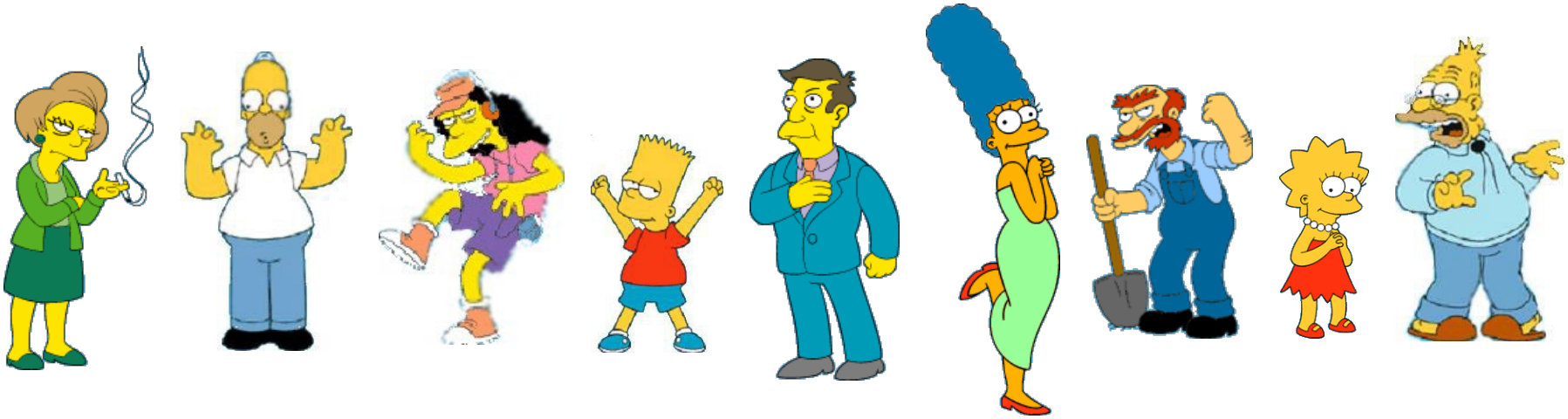
RMIT UNIVERSITY

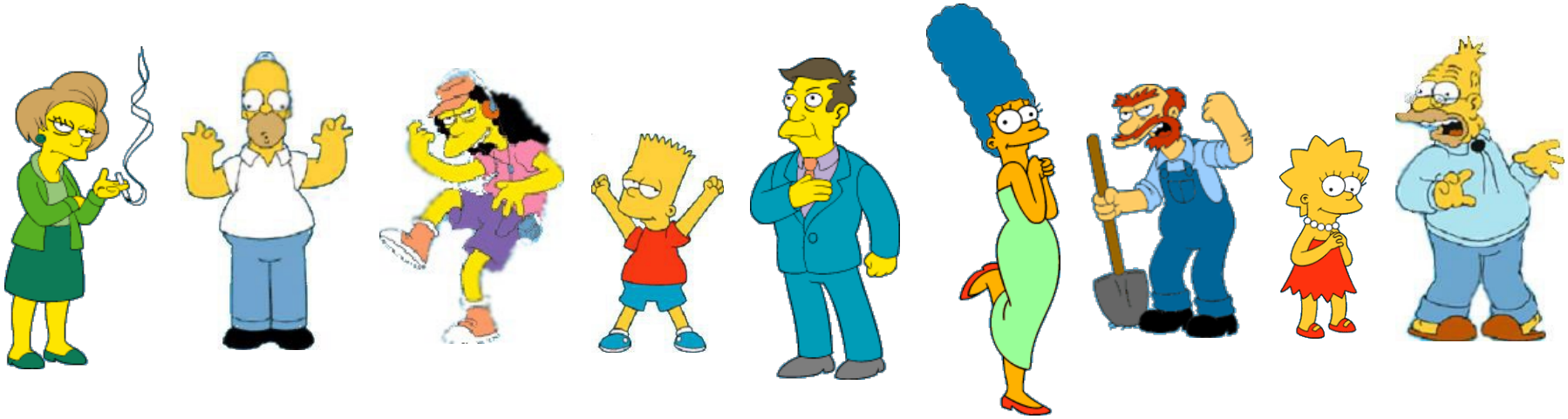
Part 1
Clustering
Distance Measure and HAC

Clustering

Arranging items into clusters

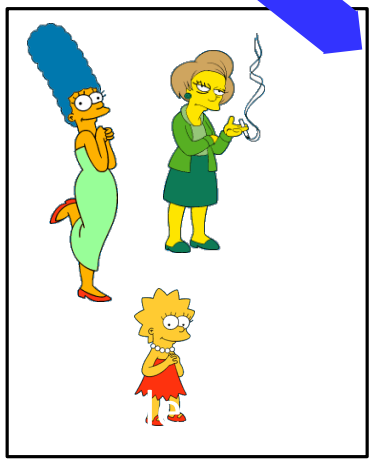
- High similarity (low distance) between objects in the same cluster
- Low similarity (high distance) between objects in different clusters



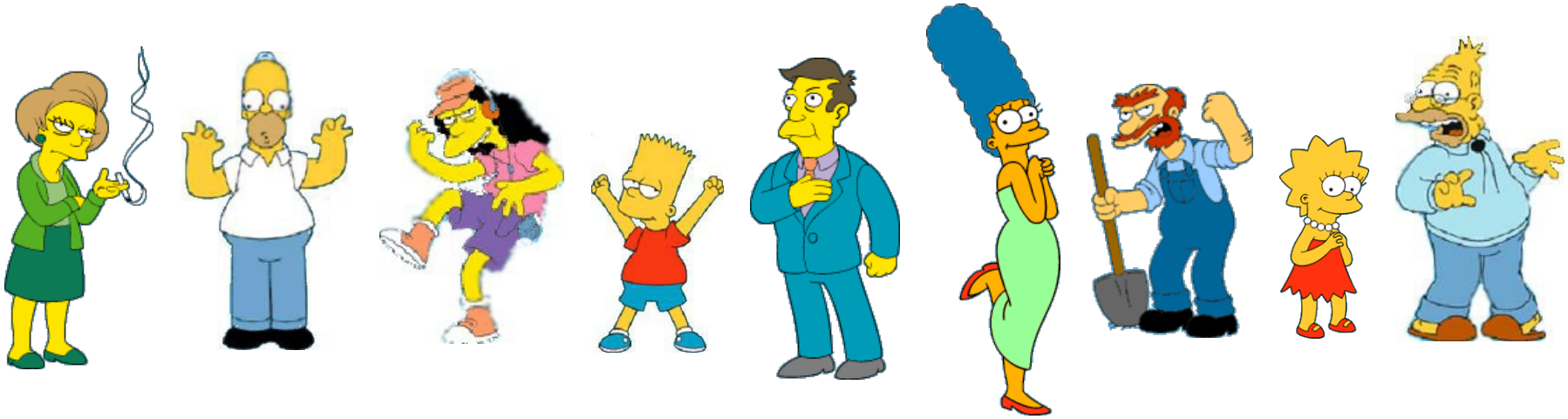


Similarity in terms of …

Gender

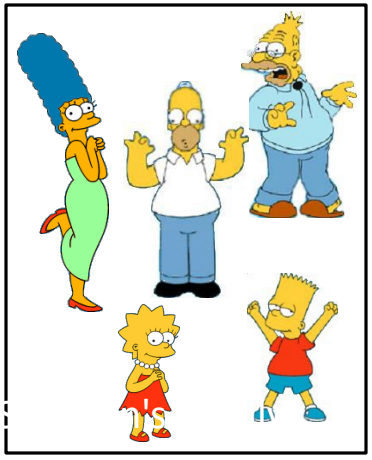


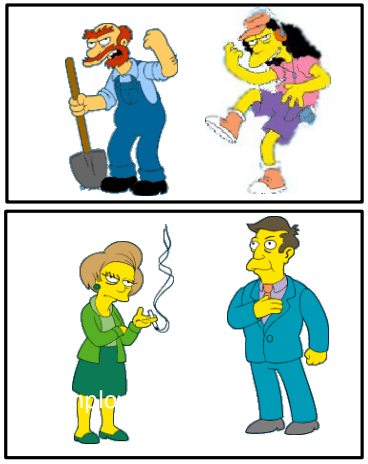


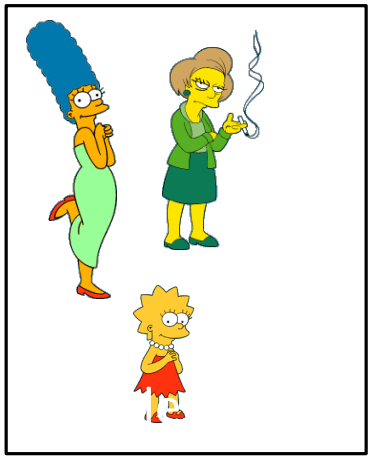


Similarity in terms of …

Family









Applications

- Exploratory analysis of large collections of objects
- Collection pre-processing for web search
- Image segmentation
- Recommender systems
- Cluster hypothesis in information retrieval
- Computational biology and bioinformatics
- Many more!

Applications

- Exploratory analysis of large
- Collection pre-processing for
- Image segmentation

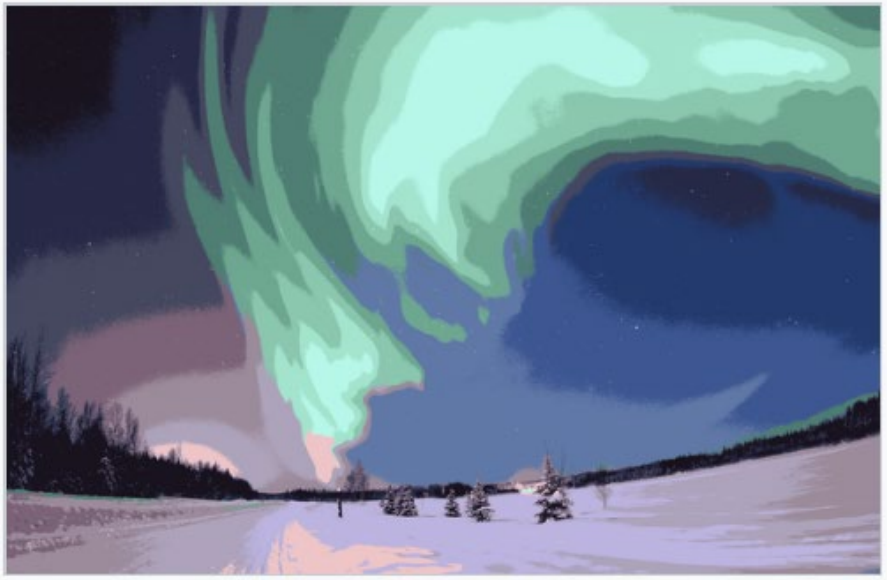
Image after running *k*-means with *k* = *16*. Note that a common technique to improve performance for large images is to downsample the image, compute the clusters, and then reassign the values to the larger image if necessary.

https://en.wikipedia.org/wiki/Image_segmentation

- Many more!



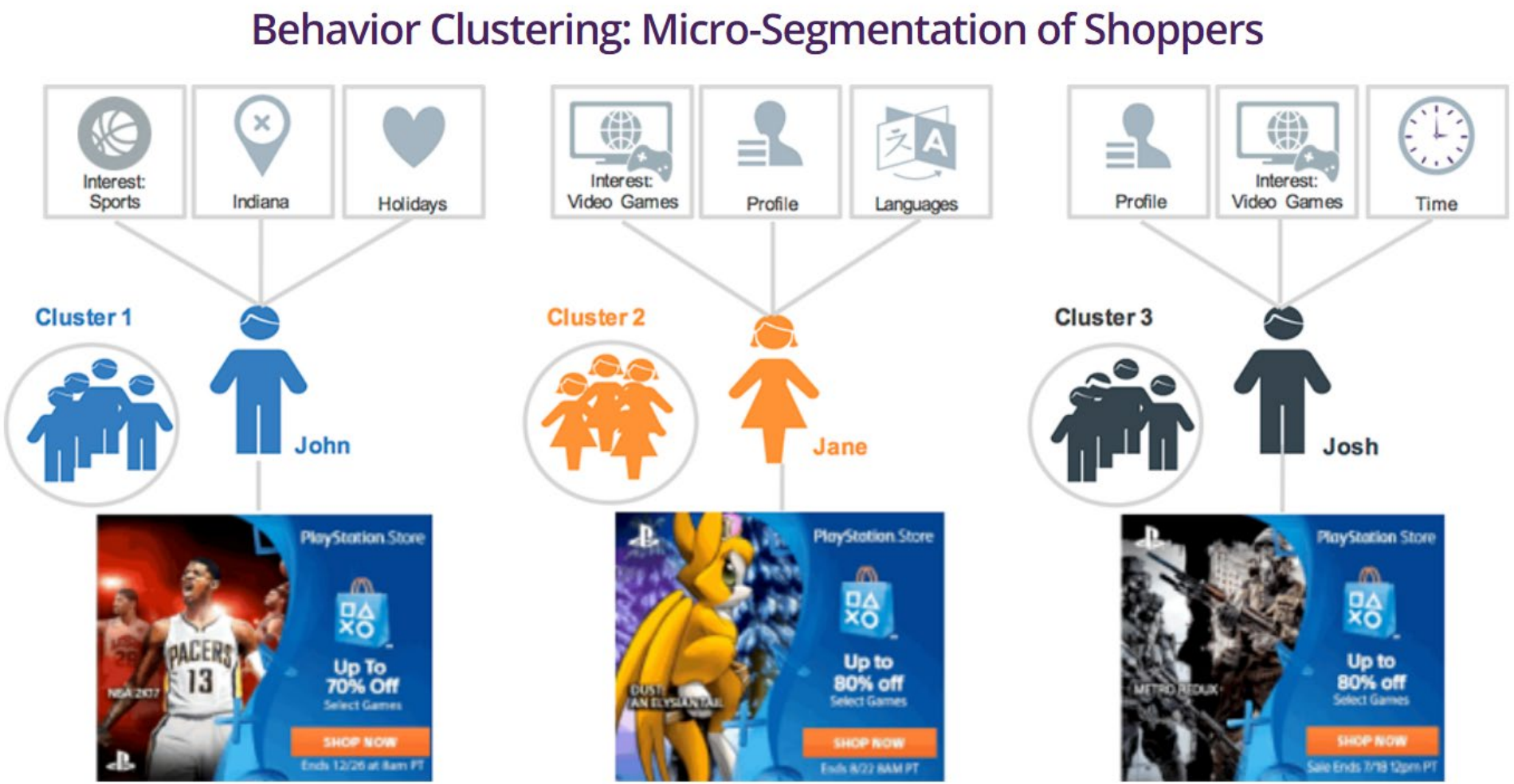
Source image.



Applications

- Exploratory analysis of large collections of objects
- Collection pre-processing for web search
- Image segmentation
- Recommender systems
- Cluster hypothesis in information retrieval
- Computational biology and bioinformatics
- Many more!

Applications



https://www.jivox.com/recommendation-engine-making-advertising-relevant/

Similarity or Distance Measure 1 - Metrics

1. Non-negativity:

$$d(x, y) \geq 0$$

2. Identity:

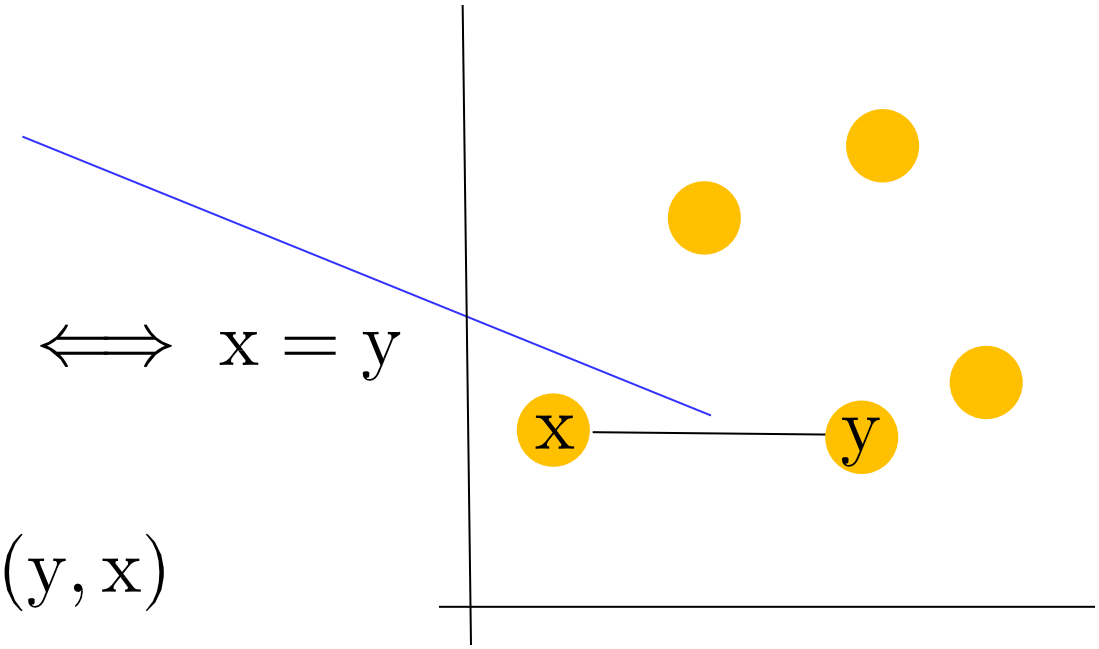
$$d(x, y) = 0 \iff x = y$$

3. Symmetry:

$$d(x, y) = d(y, x)$$

4. Triangle Inequality

$$d(x, y) \leq d(x, z) + d(z, y)$$



Similarity or Distance Measure 1 - Metrics

1. Non-negativity:

$$d(x, y) \geq 0$$

2. Identity:

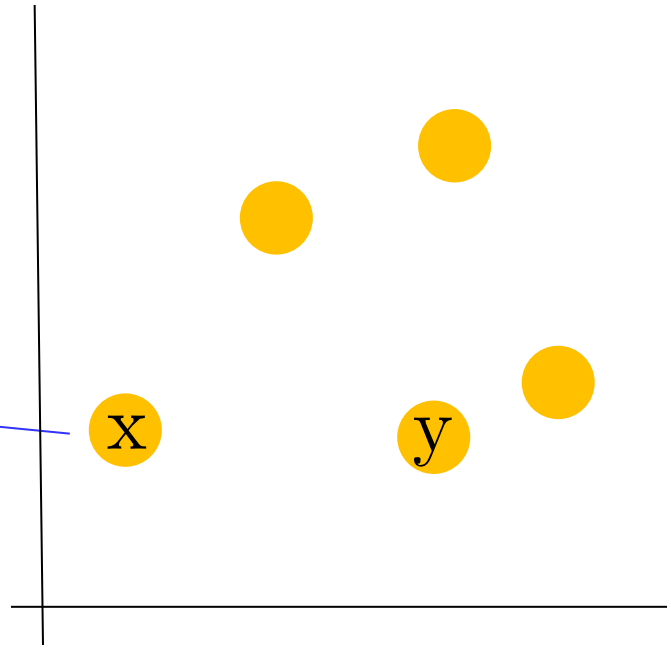
$$d(x, y) = 0 \iff x = y$$

3. Symmetry:

$$d(x, y) = d(y, x)$$

4. Triangle Inequality

$$d(x, y) \leq d(x, z) + d(z, y)$$



Similarity or Distance Measure 1 - Metrics

1. Non-negativity:

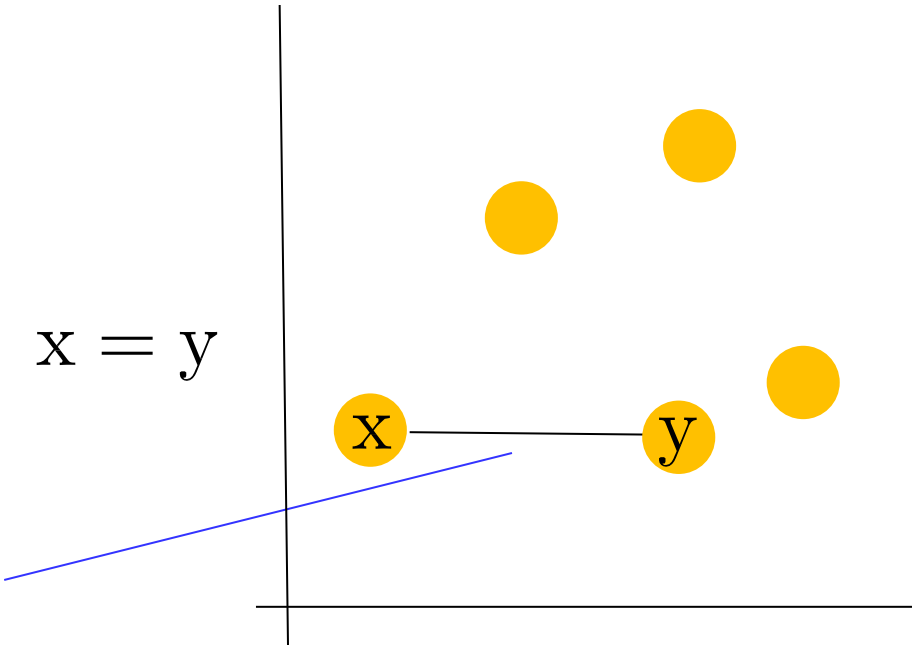
$$d(x, y) \geq 0$$

2. Identity:

$$d(x, y) = 0 \iff x = y$$

3. Symmetry:

$$d(x, y) = d(y, x)$$



4. Triangle Inequality

$$d(x, y) \leq d(x, z) + d(z, y)$$

Similarity or Distance Measure 1 - Metrics

1. Non-negativity:

$$d(x, y) \geq 0$$

2. Identity:

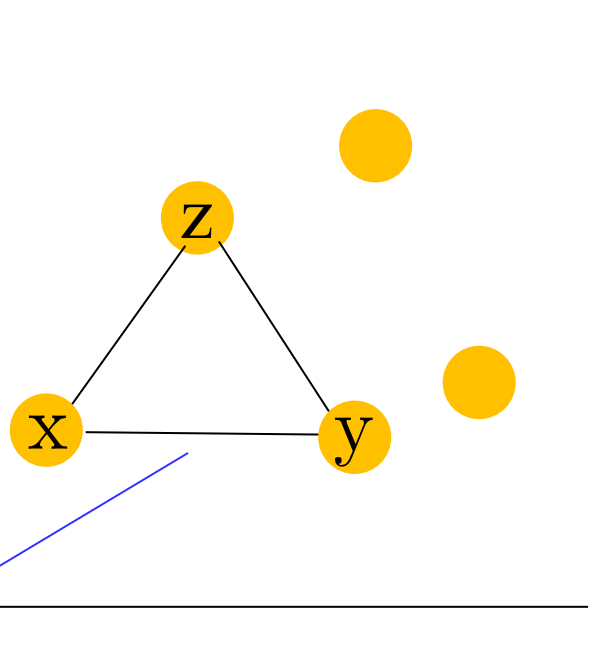
$$d(x, y) = 0 \iff x = y$$

3. Symmetry:

$$d(x, y) = d(y, x)$$

4. Triangle Inequality

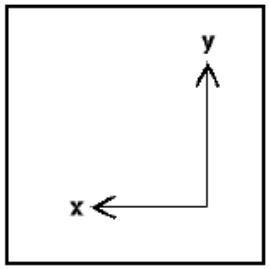
$$d(x, y) \leq d(x, z) + d(z, y)$$



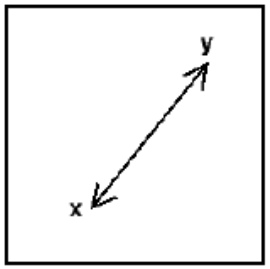
Similarity or Distance Measure 2 - Norms

Given:

$$\mathrm{x} = [x_1, x_2, \ldots x_n]$$
$$\mathrm{y} = [y_1, y_2, \ldots y_n]$$



Manhattan



Euclidean

- Manhattan distance (L_1-norm)

$$\mathrm{d}(\mathrm{x}, \mathrm{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

- Euclidean distance (L_2-norm)

$$\mathrm{d}(\mathrm{x}, \mathrm{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- L_r-norm

$$\mathrm{d}(\mathrm{x}, \mathrm{y}) = \left[\sum_{i=1}^{n} |x_i - y_i|^r \right]^{1/r}$$

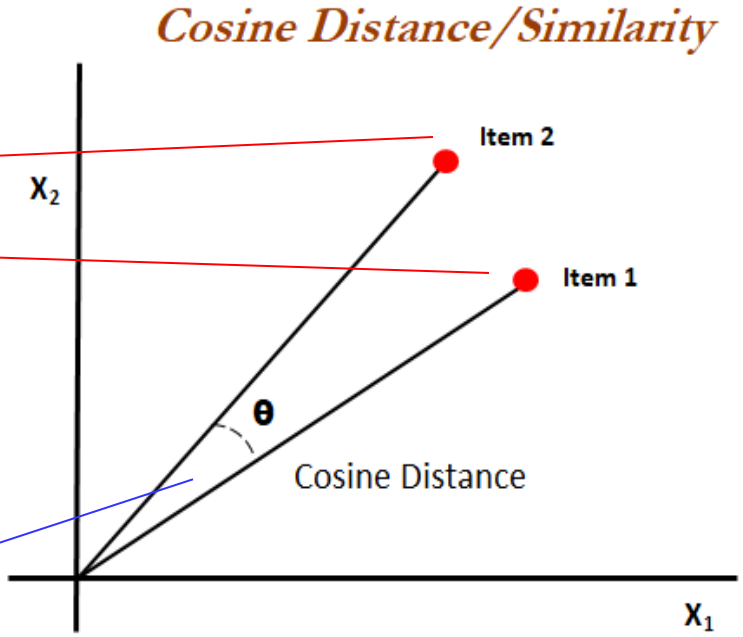


RMIT Classification: Trusted

Similarity or Distance Measure 3 - Cosine

Given:

$$\mathrm{x} = [x_1, x_2, \ldots x_n]$$
$$\mathrm{y} = [y_1, y_2, \ldots y_n]$$



- Idea: measure distance between the vectors

$$\cos\theta = \frac{\mathrm{x} \cdot \mathrm{y}}{|\mathrm{x}||\mathrm{y}|}$$

- Thus:

$$\mathrm{sim}(\mathrm{x}, \mathrm{y}) = \frac{\sum_{i=}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}}$$

RMIT Classification: Trusted

Similarity or Distance Measure 3 - Cosine

Given:

$$\mathrm{x} = [x_1, x_2, \ldots x_n]$$
$$\mathrm{y} = [y_1, y_2, \ldots y_n]$$

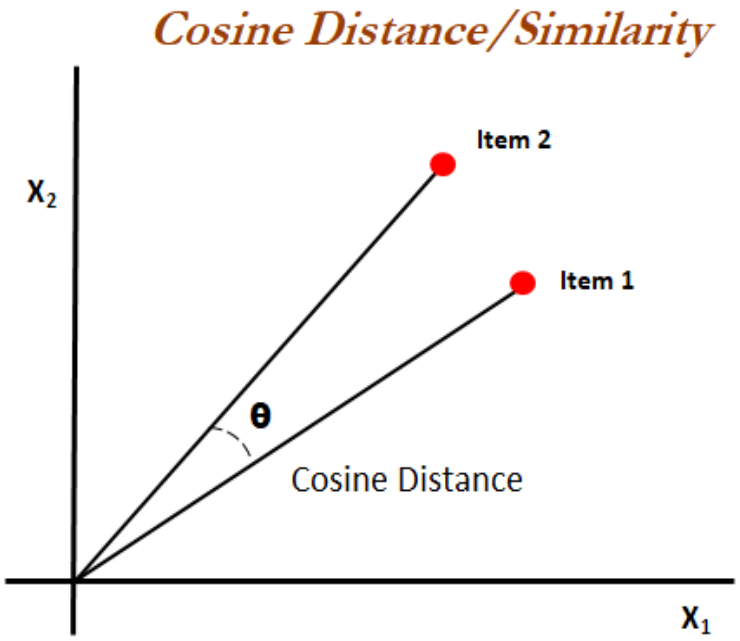
- Idea: measure distance between the vectors

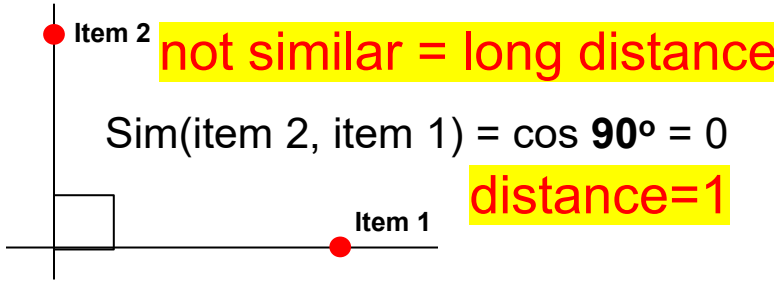
$$\cos\theta = \frac{\mathrm{x} \cdot \mathrm{y}}{|\mathrm{x}||\mathrm{y}|}$$

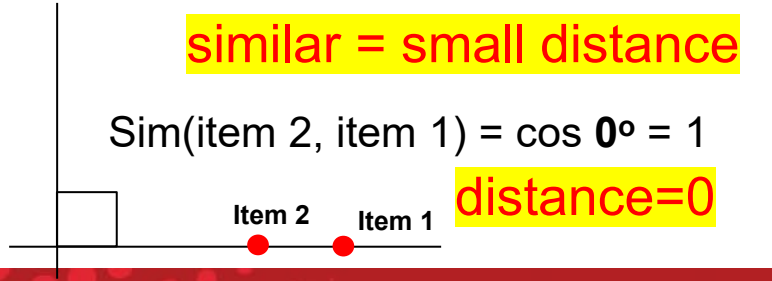
- Thus:

$$\mathrm{sim}(\mathrm{x}, \mathrm{y}) = \frac{\sum_{i=}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}}$$

$$\mathrm{d}(\mathrm{x}, \mathrm{y}) = 1 - \mathrm{sim}(\mathrm{x}, \mathrm{y})$$

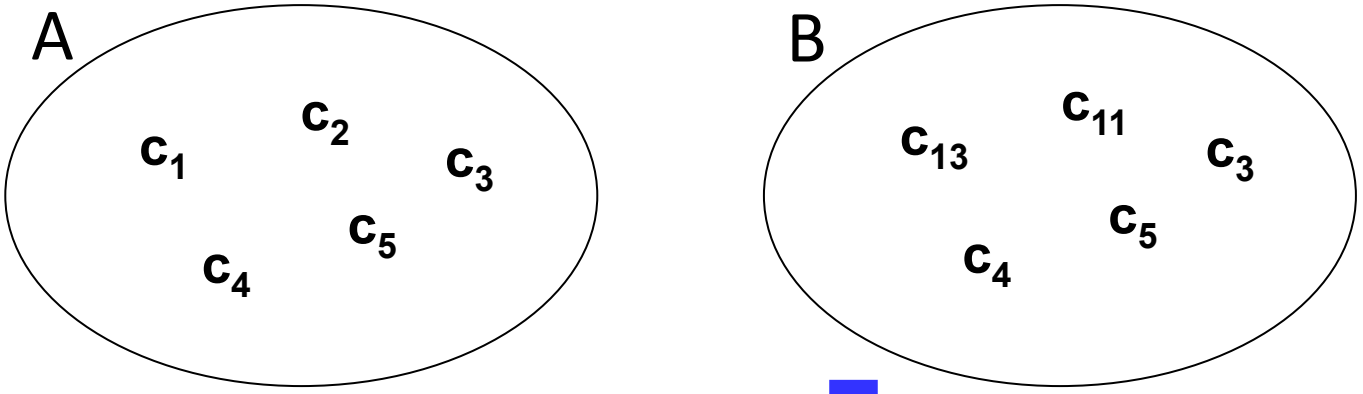




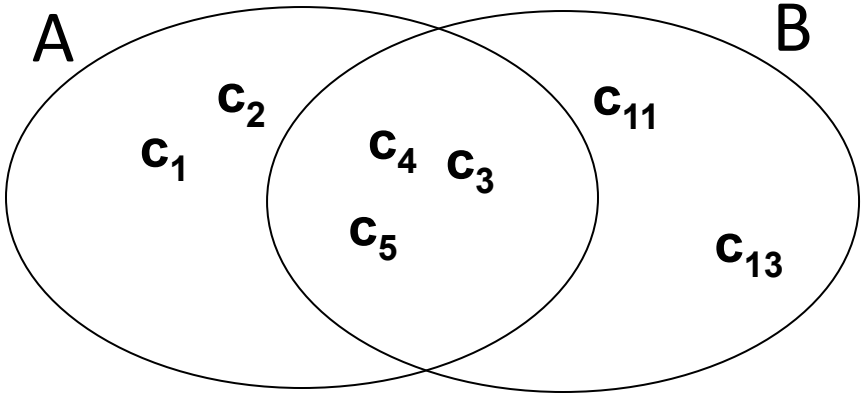


Similarity or Distance Measure 4 - Jaccard

- Given two sets A, B, Jaccard similarity:



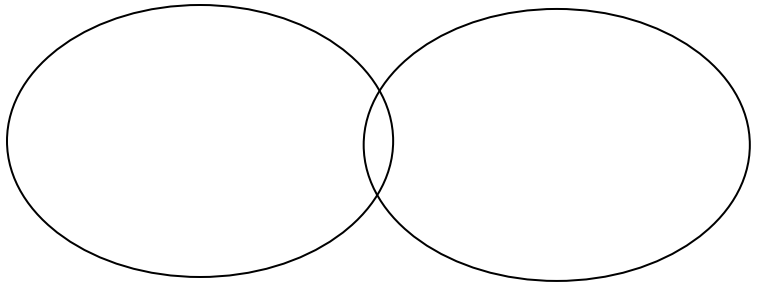
$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{3}{7}$$



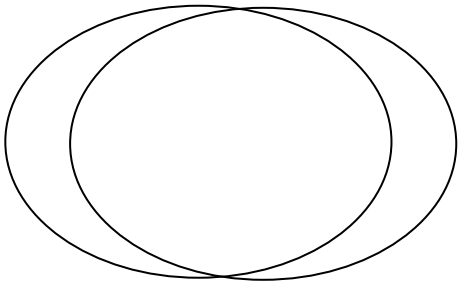
Similarity or Distance Measure 4 - Jaccard

- Given two sets A, B, Jaccard similarity:

$$\mathrm{J}(A,B) = \frac{|A \cap B|}{|A \cup B|}$$



Less similar J(A,B) → 0



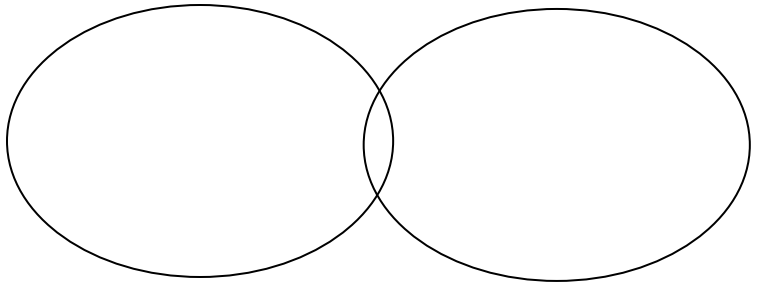
More similar J(A,B) → 1

Similarity or Distance Measure 4 - Jaccard

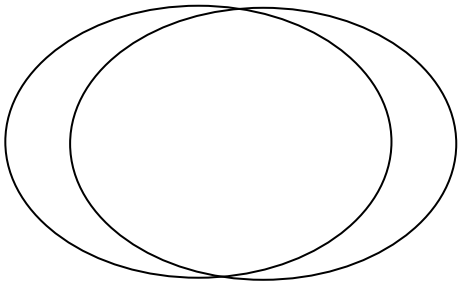
- Given two sets A, B, Jaccard similarity:

$$\mathrm{J}(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

$$\mathrm{d}(A,B) = 1 - \mathrm{J}(A,B)$$



Less similar J(A,B) → 0



More similar J(A,B) → 1

General Clustering Approaches

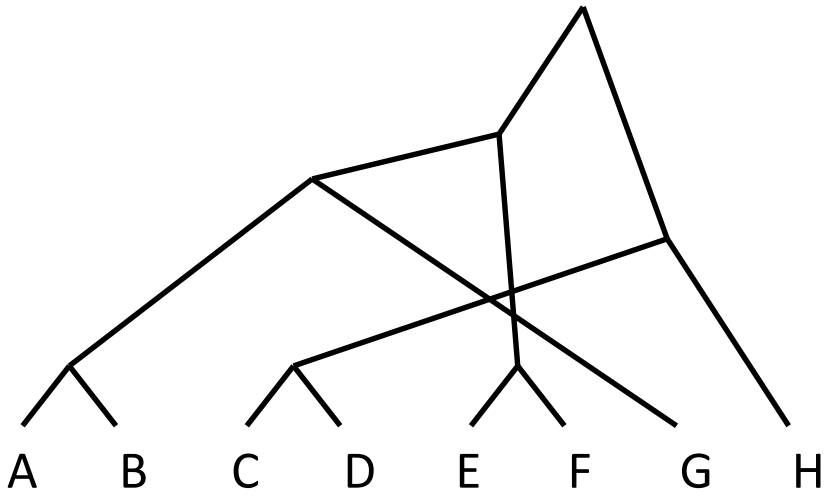
- Hierarchical Agglomerative Clustering (HAC)
  - How to use MapReduce
- *K*-Means
  - Definition
  - Lloyd's algorithm
  - MapReduce solution

Hierarchical Agglomerative Clustering (HAC)

1. Start with each document in its own cluster
2. Until there is only one cluster:
   - Find the two clusters c_i and c_j, that are most similar
   - Replace c_i and c_j with a single cluster $c_i \cup c_j$
3. The history of merges forms the hierarchy

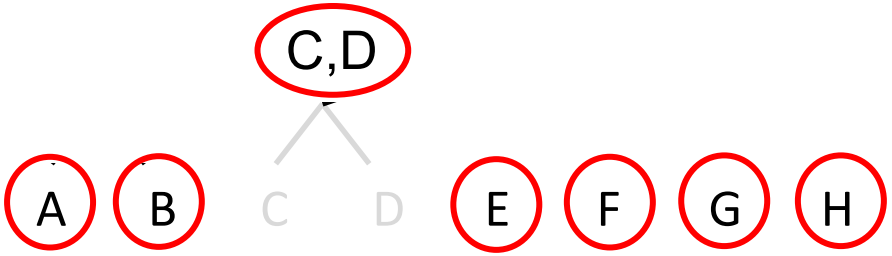
Hierarchical Agglomerative Clustering (HAC)

- Start with each document in its own cluster
- Until there is only one cluster:
  - Find the two clusters c_i and c_j, that are most similar
  - Replace c_i and c_j with a single cluster $c_i \cup c_j$
- The history of merges forms the hierarchy



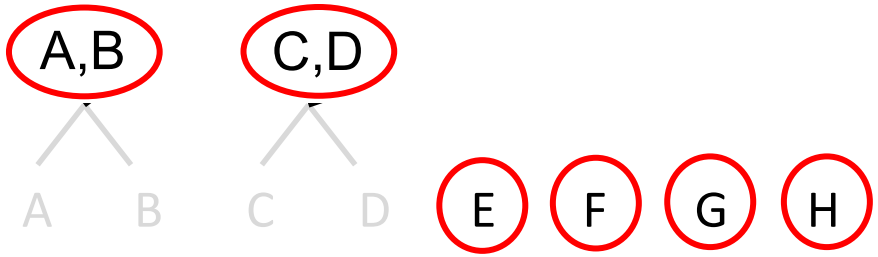
Cluster Merging

- Which two clusters do we merge?
- What's the similarity between two clusters?
  - Single Link: similarity of two most similar members
  - Complete Link: similarity of two least similar members
  - Group Average: average similarity between members



Cluster Merging

- Which two clusters do we merge?
- What's the similarity between two clusters?
  - Single Link: similarity of two most similar members
  - Complete Link: similarity of two least similar members
  - Group Average: average similarity between members



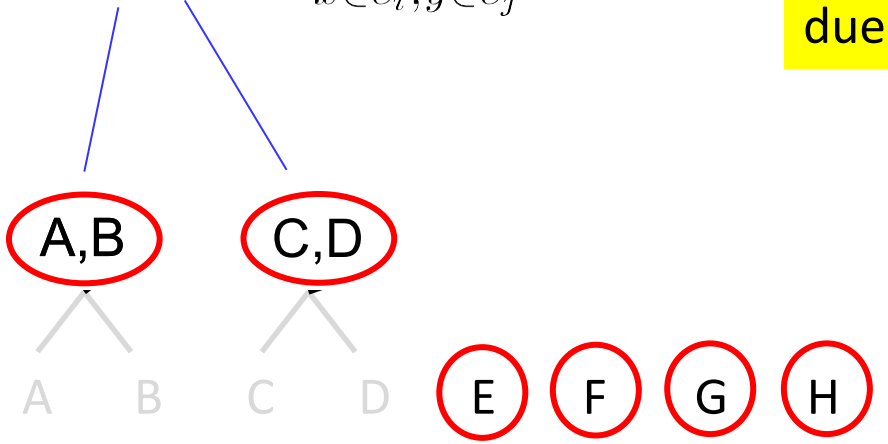
Cluster Merging

- Which two clusters do we merge?
- What's the similarity between two clusters?
  - Single Link: similarity of two most similar members
  - Complete Link: similarity of two least similar members
  - Group Average: average similarity between members

$$\text{sim}(c_i, c_j) = \max_{x \in c_i, y \in c_j} \text{sim}(x, y)$$

Can result in "straggly" (long and thin) clusters due to *chaining effect*

A, C
A, D
B, C
B, D

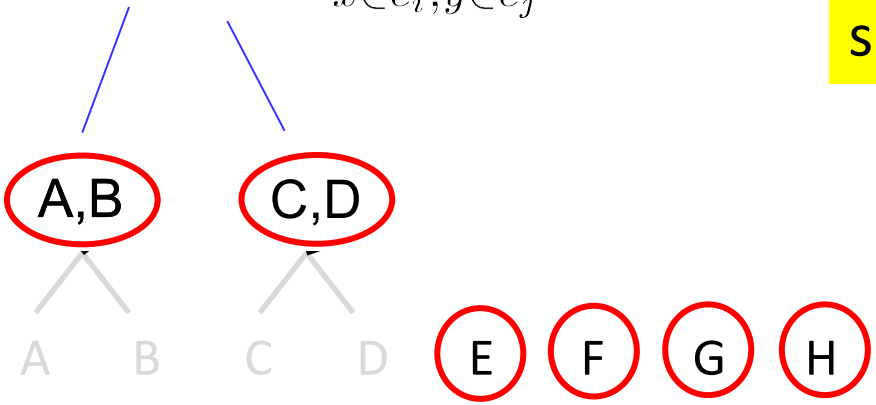


Cluster Merging

- Which two clusters do we merge?
- What's the similarity between two clusters?
  - Single Link: similarity of two most similar members
  - Complete Link: similarity of two least similar members
  - Group Average: average similarity between members

$$\text{sim}(c_i, c_j) = \min_{x \in c_i, y \in c_j} \text{sim}(x, y)$$

A, C
A, D
B, C
B, D

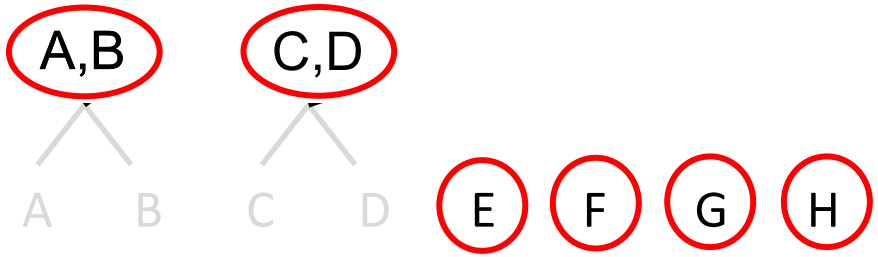


Makes more "tight" spherical clusters

Cluster Merging

- Which two clusters do we merge?
- What's the similarity between two clusters?
  - Single Link: similarity of two most similar members
  - Complete Link: similarity of two least similar members
  - Group Average: average similarity between members

A, C
A, D
B, C
B, D



End of Part 1

Part 2
K-means Clustering Algorithm and MapReduce Solution

K-means Clustering

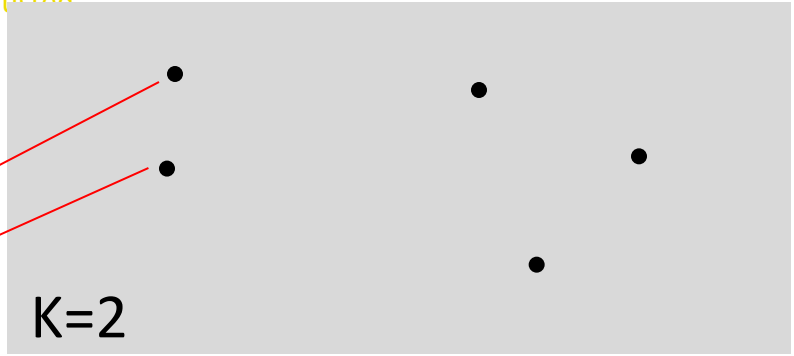
Fundamental problem in data analysis and machine learning

- Identified as one of the **top 10 algorithms in data mining** [Wu et al ’08]

Xindong Wu, Vipin Kumar, J. Ross Quinlan, Joydeep Ghosh, Qiang Yang, Hiroshi Motoda, Geoffrey J. McLachlan, Angus F. M. Ng, Bing Liu, Philip S. Yu, Zhi-Hua Zhou, Michael Steinbach, David J. Hand, Dan Steinberg: Top 10 algorithms in data mining. Knowl. Inf. Syst. 14(1): 1-37 (2008)

RMIT Classification: Trusted

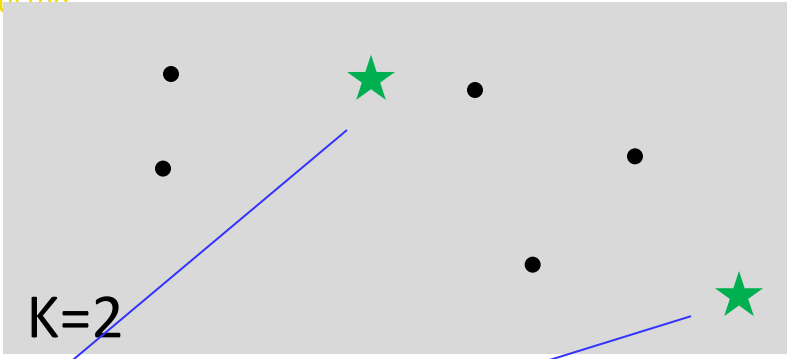
K-means Clustering



Input:

- A set $X = \{x_1, x_2, \ldots, x_n\}$ of n data points
- Number of clusters k

K-means Clustering



Input:

- A set $X = \{x_1, x_2, \ldots, x_n\}$ of n data points
- Number of clusters k

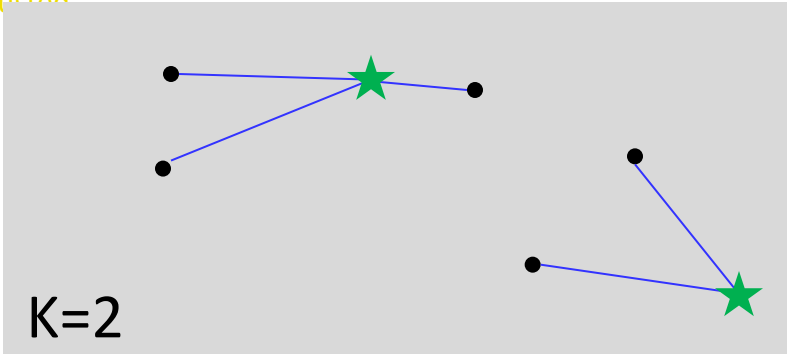
For a set $C = \{c_1, c_2, \ldots, c_k\}$ of cluster "centroids" define:

$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

where $d(x, C)$ = distance from x to the closest centroid C

RMIT Classification: Trusted

K-means Clustering



Input:

- A set $X = \{x_1, x_2, \ldots, x_n\}$ of n data points
- Number of clusters k

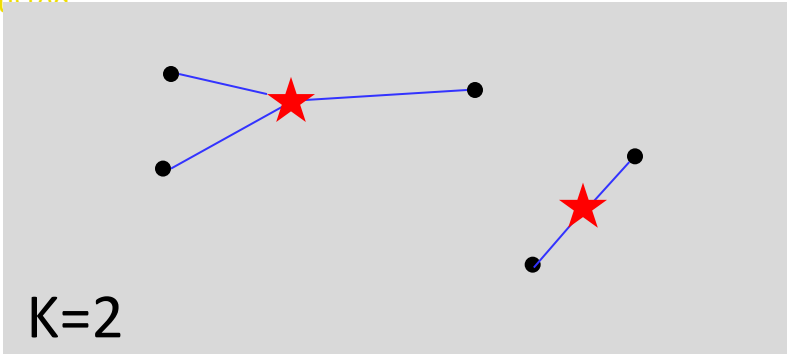
For a set $C = \{c_1, c_2, \ldots, c_k\}$ of cluster "centroid" define:

$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

where $d(x, C)$ = distance from x to the closest centroid C

RMIT Classification: Trusted

K-means Clustering



Input:

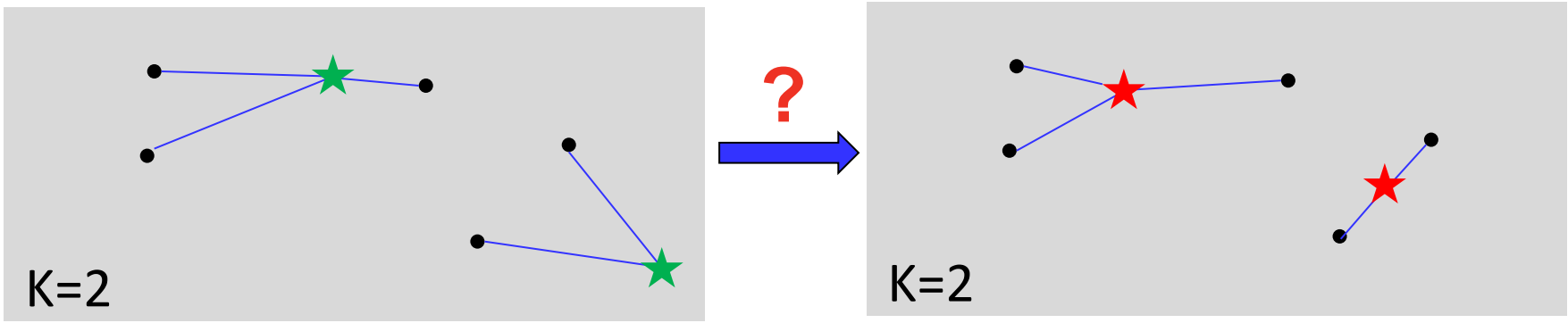
- A set $X = \{x_1, x_2, \ldots, x_n\}$ of n data points
- Number of clusters k

For a set $C = \{c_1, c_2, \ldots, c_k\}$ of cluster "centroids" define:

$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

where $d(x, C)$ = distance from x to the closest centroid C

Goal: To find a set of centroids that minimize the objective function $\varphi_X(C)$


K=2
?
K=2

Lloyd's Algorithm

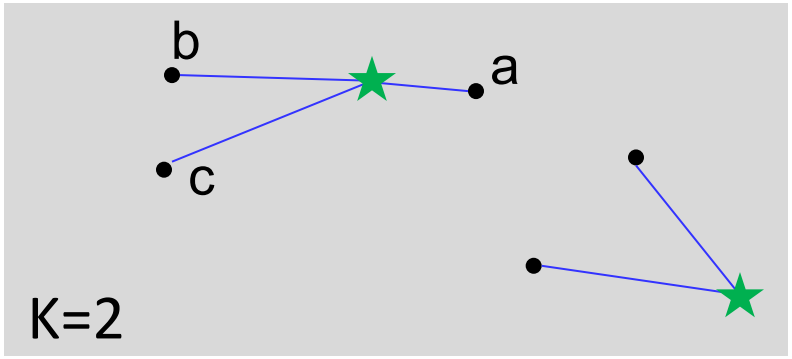
1. Select k random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

Lloyd's Algorithm

1. Select k random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

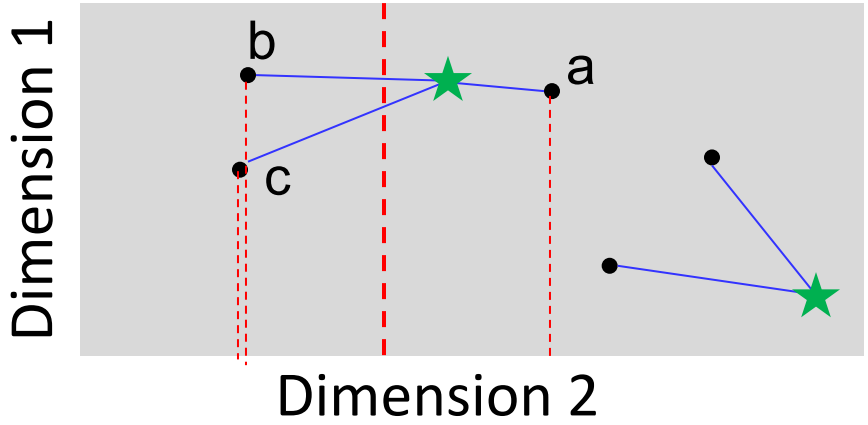


Lloyd's Algorithm

1. Select *k* random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

$$\frac{d2(a) + d2(b) + d2(c)}{3}$$

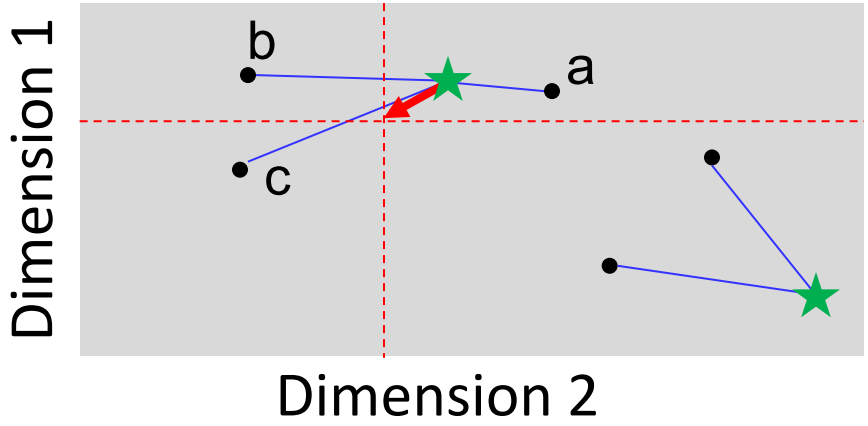


Lloyd's Algorithm

1. Select *k* random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

$$\frac{d2(a) + d2(b) + d2(c)}{3}$$



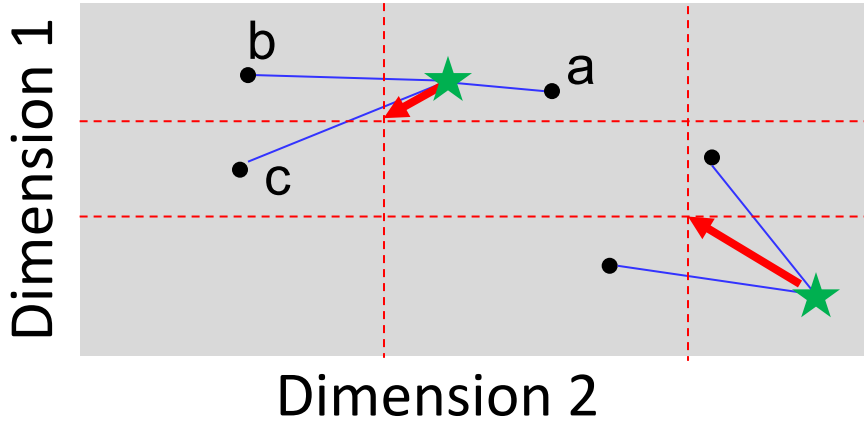
$$\frac{d1(a) + d1(b) + d1(c)}{3}$$

Lloyd's Algorithm

1. Select *k* random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

$$\frac{d2(a) + d2(b) + d2(c)}{3}$$



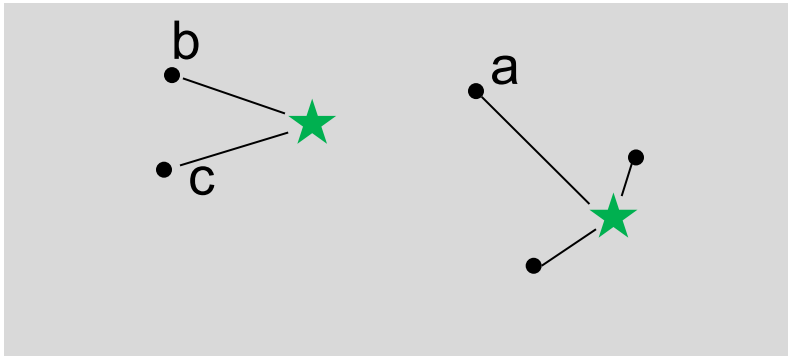
$$\frac{d1(a) + d1(b) + d1(c)}{3}$$

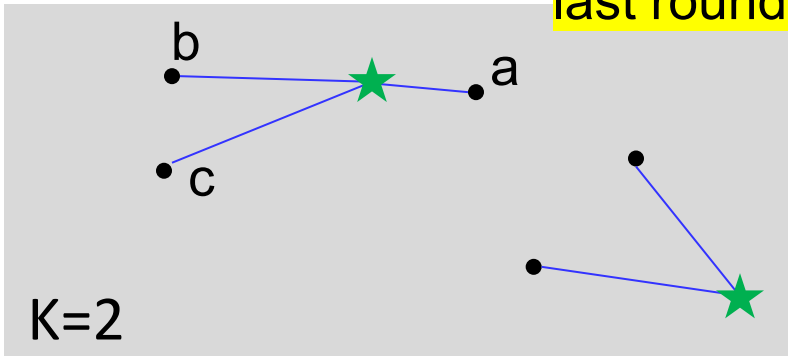
Lloyd's Algorithm

1. Select k random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
     - For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
     - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

Dimension 1

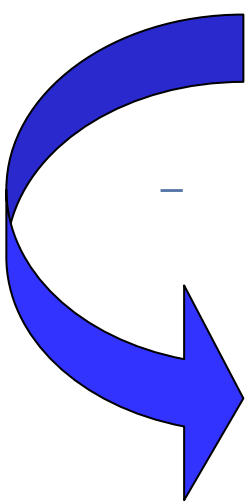




Converged => no centroid position changes from last round

Lloyd's Algorithm

1. Select *k* random point $\{c_1, c_2, \ldots c_k\}$ as seeds.
2. Take iterations until clusters converge:

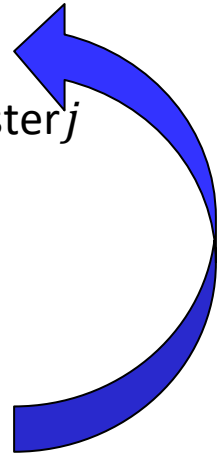


Assign each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal

- For each point x_i , find **nearest centroid** c_j , assign the point x_i to cluster j

- **Refine** the seeds to the centroid of each cluster
- For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$



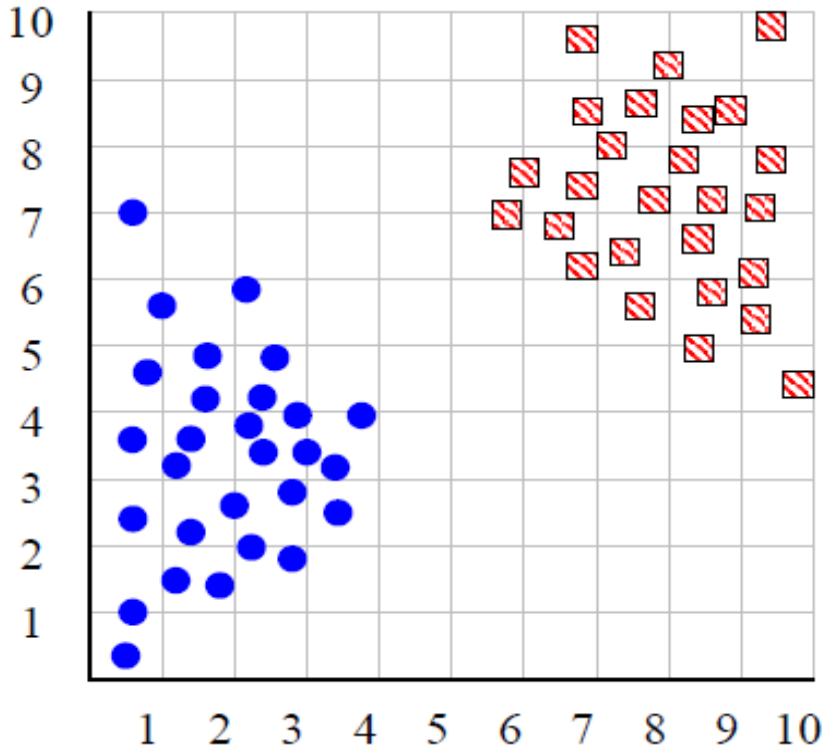
until converged

How can we tell the *right* number of clusters?

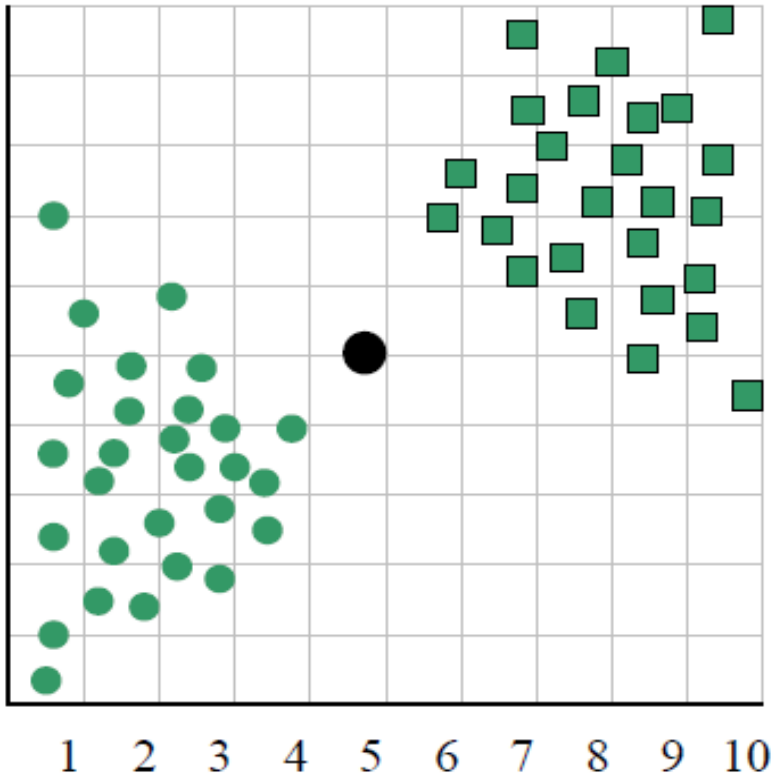
How can we tell the *right* number of clusters?

In general, this is an unsolved problem.

- However there are many approximate methods. In the next few slides we will see the following example.

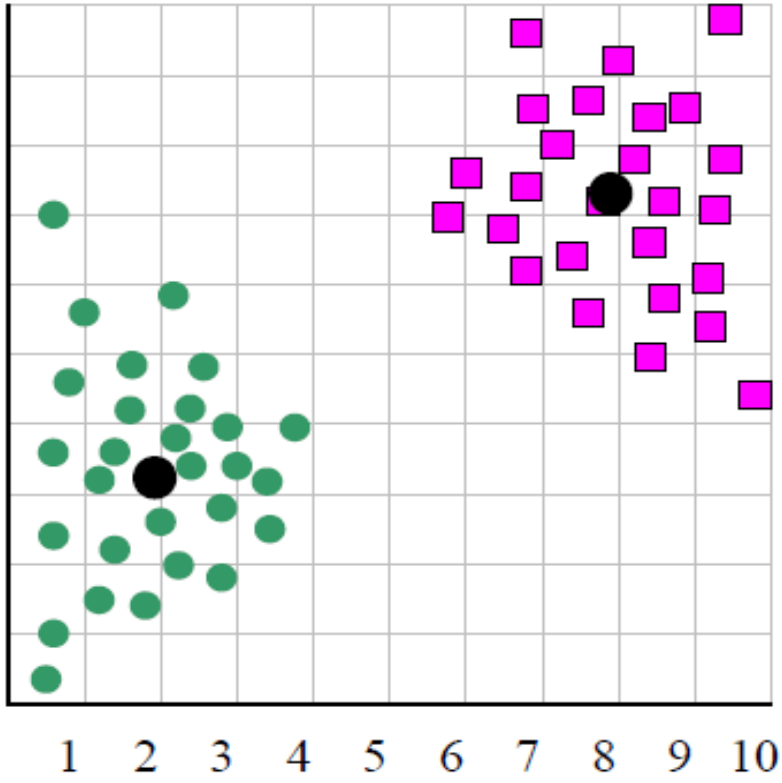


When k = 1, the objective function is 873.0



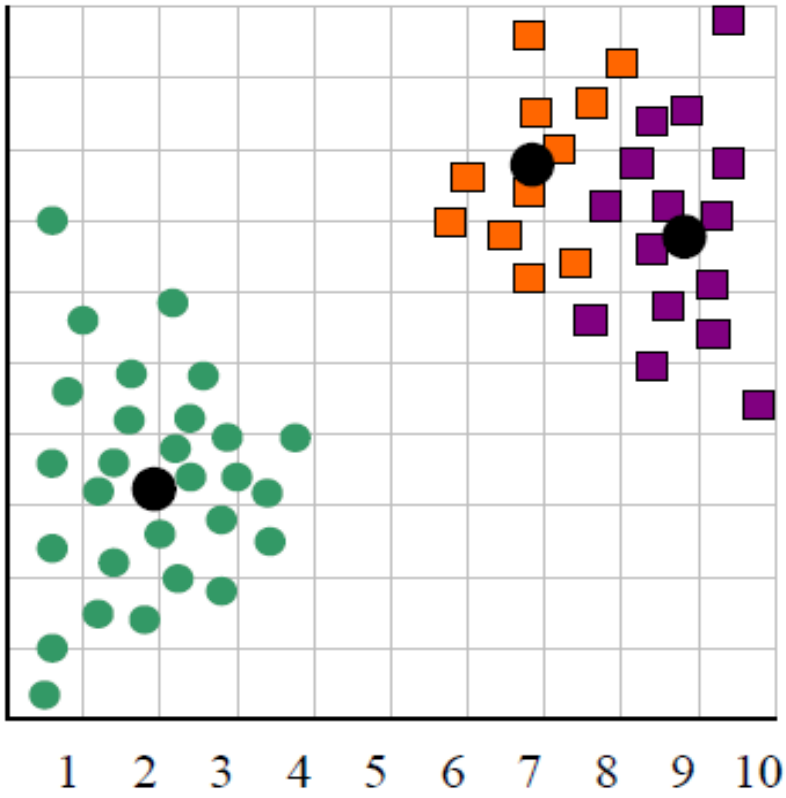
$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

When k = 2, the objective function is 173.1



$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

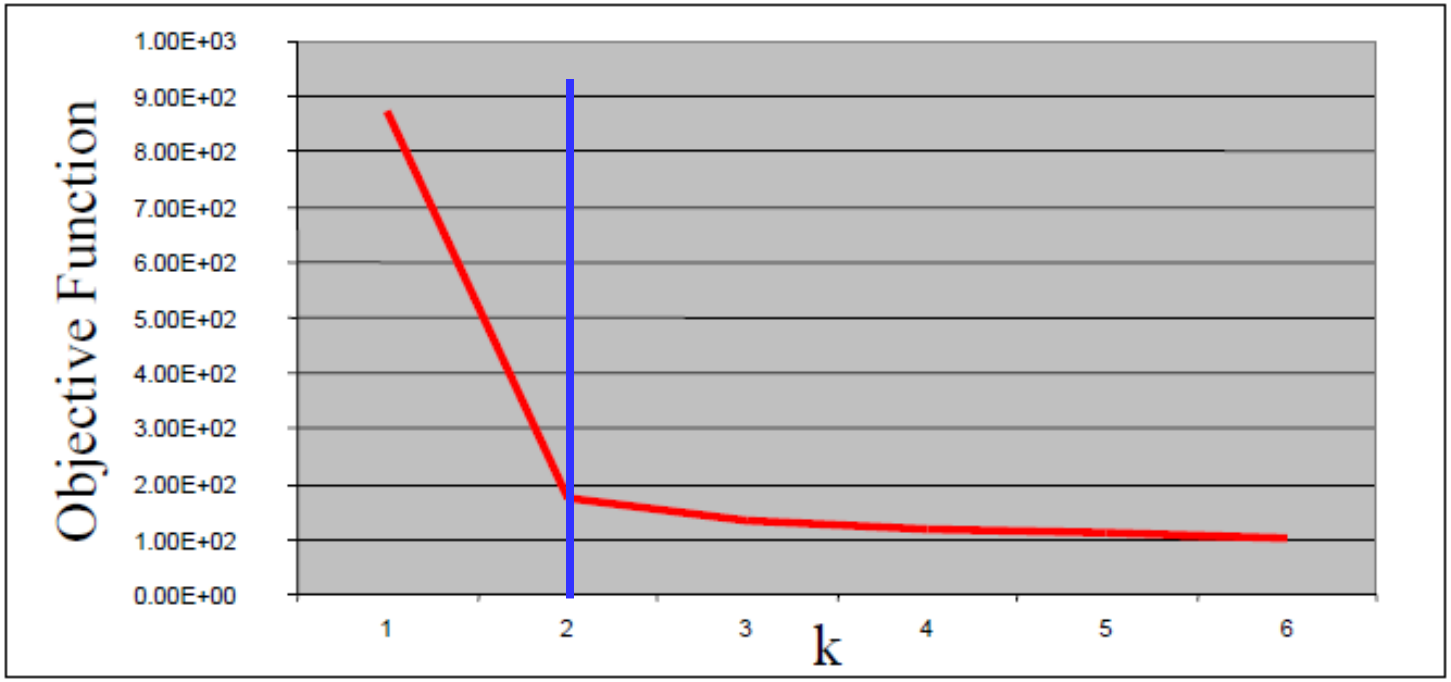
When k = 3, the objective function is 133.6



$$\varphi_X(C) = \sum_{x \in X} d(x, C)^2$$

We can plot the objective function values for k equals 1 to 6…

- The abrupt change at k = 2, is highly suggestive of two clusters in the data.
- This technique for determining the number of clusters is known as "knee finding" or "elbow finding".



Pseudocode

mapper.py

Input: point file

$c \leftarrow LoadClusters()$

For each point p in point file

$n \leftarrow NearestClusterID$(clusters c, point p)

$p \leftarrow ExtendPoint$(point p)

emit(clusterid n, point p)

Load current k centroids

Allocate each point to the nearest centroid

p(x=3, y=7) -> p(3, 7, 1) why?

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$

$s \leftarrow InitPointSum\ ()$

For each point $p \in [p_1, p_2, \dots]$ do

$s \leftarrow s + p$

$m \leftarrow ComputeCentroid$ (point s)

emit(clusterid n, point p)

Each point is extended by one dimension and initialized to a value of one to represent the count for cluster normalization (e.g., x, y dimensions + 1 more dimension for count # of points in this cluster – for computing mean value in each dimension – note necessary for location aggregation)

Pseudocode

mapper.py

Input: point file
$c \leftarrow LoadClusters()$
For each point p in point file
    $n \leftarrow NearestClusterID$(clusters c, point p)
    $p \leftarrow ExtendPoint$(point p)
    emit(clusterid n, point p)

In each cluster, for each dimension, the values of all points are summed.

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$
$s \leftarrow InitPointSum\ ()$
For each point $p \in [p_1, p_2, \dots]$ do
    $s \leftarrow s + p$
$m \leftarrow ComputeCentroid$ (point s)
emit(clusterid n, point p)

In each cluster, for each dimension, compute the average of the sum.

centroid ID

New location of the centroid

mapper.py

Input: point file

$c \leftarrow LoadClusters()$

For each point p in point file

$n \leftarrow NearestClusterID$(clusters c, point p)

$p \leftarrow ExtendPoint$(point p)

emit(clusterid n, point p)

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$

$s \leftarrow InitPointSum\ ()$

For each point $p \in [p_1, p_2, \dots]$ do

$s \leftarrow s + p$

$m \leftarrow ComputeCentroid$ (point s)

emit(clusterid n, point p)

Lloyd's Algorithm

1. Select k random point $\{s_1, s_2, \dots s_k\}$ as seeds.
2. Until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
   - For each point x_i , find **nearest centroid** c_j, assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
   - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

If not converged

RMIT Classification: Trusted

Pseudocode - with in-mapper combining

mapper.py

Input: point file
$c \leftarrow LoadClusters()$
$H \leftarrow a\ new\ list$
For each point p in point file
    $n \leftarrow NearestClusterID$(clusters c, point p)
    $p \leftarrow ExtendPoint$(point p)
    $H\{n\} \leftarrow H\{n\} + p$
For each clustered $n \in H$ do
    emit(clusterid n, point $H\{n\}$)

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$
$s \leftarrow InitPointSum\ ()$
For each point $p \in [p_1, p_2, \dots]$ do
    $s \leftarrow s + p$
$m \leftarrow ComputeCentroid$ (point s)
emit(clusterid n, point p)

RMIT Classification: Trusted

With in-mapper combining

mapper.py

Input: point file
$c \leftarrow LoadClusters()$
$H \leftarrow a\ new\ list$
For each point p in point file
  $n \leftarrow NearestClusterID$(clusters c, point p)
  $p \leftarrow ExtendPoint$(point p)
  $H\{n\} \leftarrow H\{n\} + p$
For each clustered $n \in H$ do
  emit(clusterid n, point $H\{n\}$)

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$
$s \leftarrow InitPointSum\ ()$
For each point $p \in [p_1, p_2, \dots]$ do
  $s \leftarrow s + p$
$m \leftarrow ComputeCentroid$ (point s)
emit(clusterid n, point p)

No in-mapper combining

mapper.py

Input: point file
$c \leftarrow LoadClusters()$
For each point p in point file
  $n \leftarrow NearestClusterID$(clusters c, point p)
  $p \leftarrow ExtendPoint$(point p)
  emit(clusterid n, point p)

reducer.py

Input: clusterid n, points $[p_1, p_2, \dots]$
$s \leftarrow InitPointSum\ ()$
For each point $p \in [p_1, p_2, \dots]$ do
  $s \leftarrow s + p$
$m \leftarrow ComputeCentroid$ (point s)
emit(clusterid n, point p)

Each cluster maintains a list H

RMIT Classification: Trusted

With in-mapper combining

mapper.py

Input: point file

$c \leftarrow LoadClusters()$

$H \leftarrow a\ new\ list$

For each point p in point file

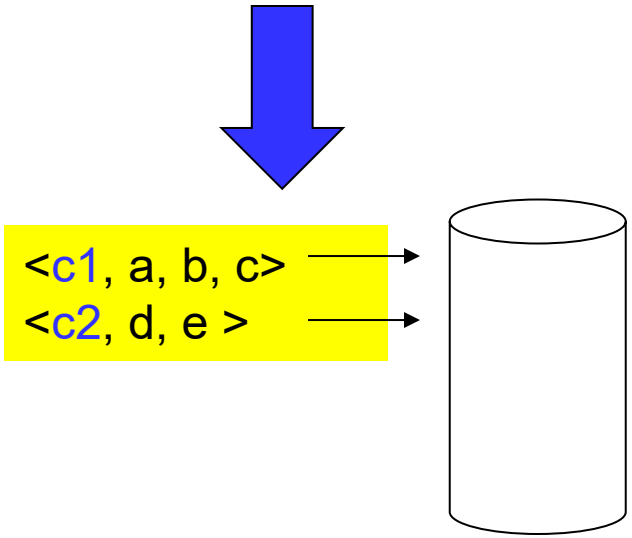
$n \leftarrow NearestClusterID$(clusters c, point p)

$p \leftarrow ExtendPoint$(point p)

$H\{n\} \leftarrow H\{n\} + p$

For each clustered $n \in H$ do

emit(clusterid n, point $H\{n\}$)



No in-mapper combining

mapper.py

Input: point file

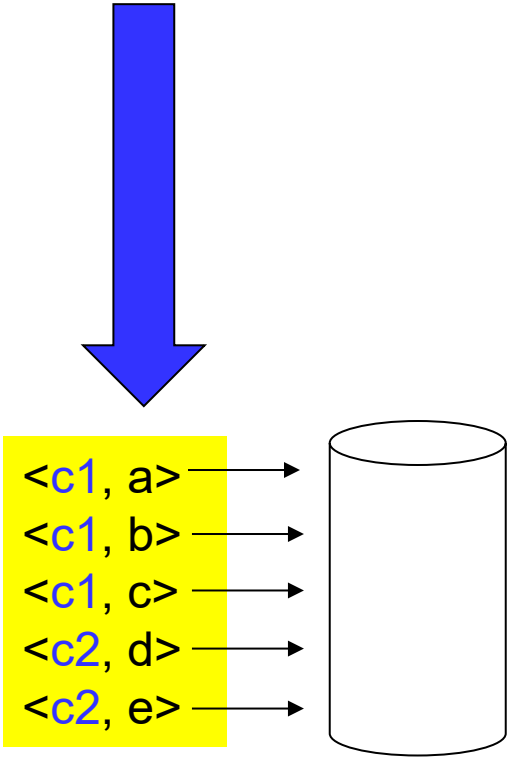
$c \leftarrow LoadClusters()$

For each point p in point file

$n \leftarrow NearestClusterID$(clusters c, point p)

$p \leftarrow ExtendPoint$(point p)

emit(clusterid n, point p)



End of Part 2

Part 3

MapReduce Python Programming

"programming case study - kmeans"